

SKILLWEEK5

TASK1: To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation

```
rishill10@ASUS16X: ~/Skill5  ×  +  ∨

rishill10@ASUS16X:~$ mkdir Skill5
cd Skill5
rishill10@ASUS16X:~/Skill5$ nano task1.c
rishill10@ASUS16X:~/Skill5$ gcc task1.c -o task1
./task1
Current directory: /home/rishill10/Skill5
Changing to /tmp...
New directory: /tmp
Previous directory: /home/rishill10/Skill5
rishill10@ASUS16X:~/Skill5$ |
```

```
rishill10@ASUS16X: ~/Skill5  ×  +  ∨

GNU nano 8.7.1 task1.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <limits.h>

int main() {
    char current[PATH_MAX];
    char previous[PATH_MAX];

    getcwd(current, sizeof(current));
    strcpy(previous, current);

    printf("Current directory: %s\n", current);
    printf("Changing to /tmp...\n");

    if (chdir("/tmp") == 0) {
        strcpy(previous, current);
        getcwd(current, sizeof(current));
        printf("New directory: %s\n", current);
        printf("Previous directory: %s\n", previous);
    } else {
        perror("cd failed");
    }
    return 0;
}
```

TASK2: To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

```
rishill10@ASUS16X: ~/Skill5  ×  +  ∨  
rishill10@ASUS16X:~/Skill5$ nano task2.c  
rishill10@ASUS16X:~/Skill5$ gcc task2.c -o myshell  
./myshell  
myshell> pwd  
/home/rishill10/Skill5  
myshell> cd /tmp  
myshell> pwd  
/tmp  
myshell> hello  
myshell: invalid command  
myshell> exit  
Exiting shell...  
rishill10@ASUS16X:~/Skill5$ |
```

```
rishill10@ASUS16X: ~/Skills  ×  +  ▾
GNU nano 8.7.1 task2.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

void pwd_command() {
    char path[1024];
    if (getcwd(path, sizeof(path)) != NULL)
        printf("%s\n", path);
    else
        perror("pwd");
}

void cd_command(char *path) {
    if (path == NULL) {
        printf("cd: missing argument\n");
        return;
    }
    if (chdir(path) != 0)
        perror("cd");
}

void exit_command() {
    printf("Exiting shell...\n");
    exit(0);
}

int main() {
    char input[100];
    char *command;
    char *argument;

    while (1) {
        printf("myshell> ");
        fgets(input, sizeof(input), stdin);
        input[strcspn(input, "\n")] = '\0'; // Remove newline

        command = strtok(input, " ");
        argument = strtok(NULL, " ");

        if (command == NULL) continue;

        if (strcmp(command, "pwd") == 0) {
            pwd_command();
        } else if (strcmp(command, "cd") == 0) {
            cd_command(argument);
        } else if (strcmp(command, "exit") == 0) {
            exit_command();
        } else {
            printf("myshell: invalid command\n");
        }
    }
    return 0;
}
```

```
        if (strcmp(command, "pwd") == 0) {
            pwd_command();
        } else if (strcmp(command, "cd") == 0) {
            cd_command(argument);
        } else if (strcmp(command, "exit") == 0) {
            exit_command();
        } else {
            printf("myshell: invalid command\n");
        }
    }
    return 0;
}
```

TASK3: To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior

```
rishill10@ASUS16X: ~/Skill5  ×  +  v

rishill10@ASUS16X:~/Skill5$ nano task3.c
rishill10@ASUS16X:~/Skill5$ gcc task3.c -o task3
./task3
Testing pwd command:
Current Directory: /home/rishill10/Skill5

Testing exit request...
Resources cleaned successfully.
Program exiting...
rishill10@ASUS16X:~/Skill5$ |
```

```
rishill10@ASUS16X: ~/Skill5  ×  +  v

GNU nano 8.7.1 task3.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <limits.h>

void show_current_directory() {
    char path[PATH_MAX];
    if (getcwd(path, sizeof(path)) != NULL) {
        printf("Current Directory: %s\n", path);
    } else {
        perror("getcwd");
    }
}

int main() {
    printf("Testing pwd command:\n");
    show_current_directory();

    printf("\nTesting exit request...\n");
    printf("Resources cleaned successfully.\n");
    printf("Program exiting...\n");

    return 0;
}|
```

TASK4: To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality

```
rishill10@ASUS16X: ~/Skill5  ×  +  ∨  
rishill10@ASUS16X:~/Skill5$ nano task4.c  
rishill10@ASUS16X:~/Skill5$ gcc task4.c -o task4  
./task4  
Variable exported successfully.  
STUDENT = Rishil  
rishill10@ASUS16X:~/Skill5$ |
```

```
rishill10@ASUS16X: ~/Skill5  ×  +  ∨  
GNU nano 8.7.1 task4.c  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
  
int main() {  
    char variable[] = "STUDENT";  
    char value[] = "Jhansi";  
  
    if (setenv(variable, value, 1) == 0) {  
        printf("Variable exported successfully.\n");  
    } else {  
        perror("setenv");  
        return 1;  
    }  
  
    printf("STUDENT = %s\n", getenv("STUDENT"));  
    return 0;  
}
```

```
rishill10@ASUS16X: ~/Skill5  ×  +  ∨  
rishill10@ASUS16X:~/Skill5$ nano task4.c  
rishill10@ASUS16X:~/Skill5$ gcc task4.c -o task4  
./task4  
Variable exported successfully.  
STUDENT = Rishil  
rishill10@ASUS16X:~/Skill5$ nano child.c  
rishill10@ASUS16X:~/Skill5$ nano parent.c  
rishill10@ASUS16X:~/Skill5$ gcc child.c -o child  
gcc parent.c -o parent  
rishill10@ASUS16X:~/Skill5$ ./parent  
Parent: STUDENT = Rishil  
Child process received STUDENT = Rishil  
rishill10@ASUS16X:~/Skill5$ |
```

```
rishill10@ASUS16X: ~/Skill5  ×  +  ∨  
GNU nano 8.7.1 child.c  
#include <stdio.h>  
#include <stdlib.h>  
  
int main() {  
    char *value = getenv("STUDENT");  
    if (value != NULL)  
        printf("Child process received STUDENT = %s\n", value);  
    else  
        printf("STUDENT variable not found\n");  
    return 0;  
}|
```



rishill10@ASUS16X: ~/Skill5



GNU nano 8.7.1

parent.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    setenv("STUDENT", "Rishil", 1);
    printf("Parent: STUDENT = %s\n", getenv("STUDENT"));

    pid_t pid = fork();

    if (pid == 0) {
        // Child process
        execl("./child", "child", NULL);
        perror("execl");
        exit(1);
    }

    wait(NULL); // Parent waits for child
    return 0;
}
```